AWS: Global Infrastructure

Tuesday, 19 May 2026 3:02 PM

Topics we are going to cover in this:

AWS Global Infrastructure:

- Region
- Availability Zones (AZ)
- Local Zone
- Edge Location (PoP i.e Point of Presence)

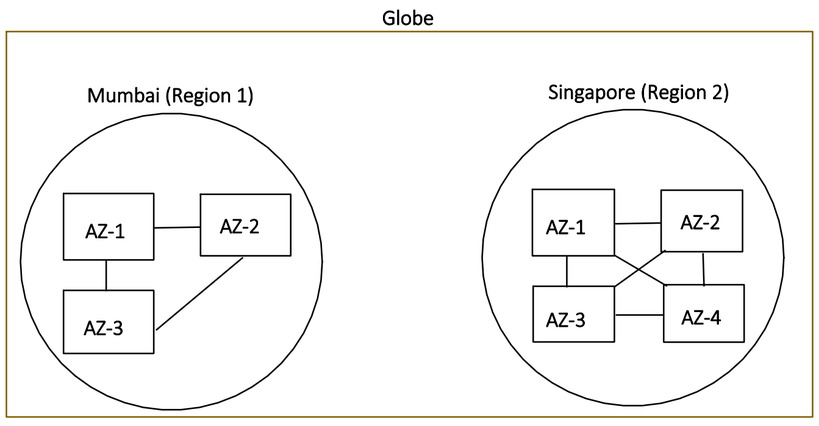
Disaster Recovery Metrics:
- RP0
- RTO

Disaster Recover Strategies:
- Backup and Restore
- Pilot Light
- Active-Passive (Warm Standby)
- Active-Active

AWS Global Infrastructure:

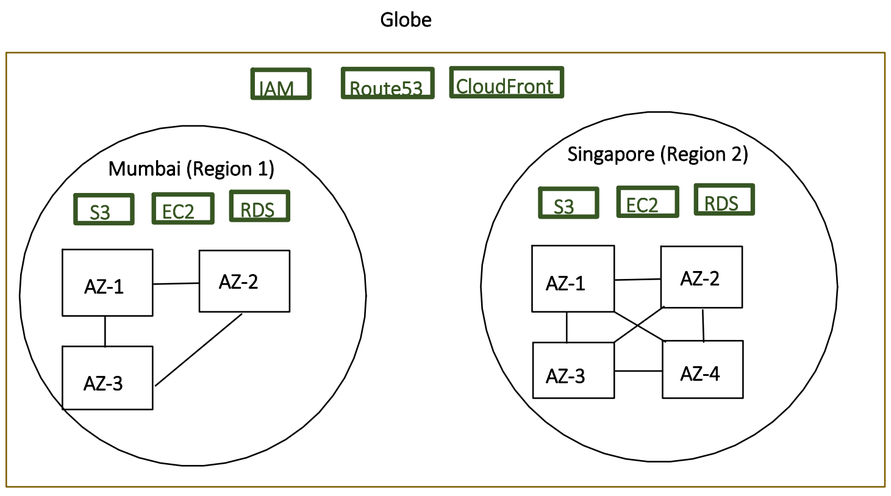
1. Region

Consider it as a City (geographic area) where AWS has built a cluster of Availability Zones.



- Many AWS services are regional, and data does not automatically replicate across regions unless explicitly configured.
  Example:
  Regional Services: means services created in Region1, other Regions can not automatically see it.
  - EC2,
  - RDS,
  - Lambda
    etc. are regional services.

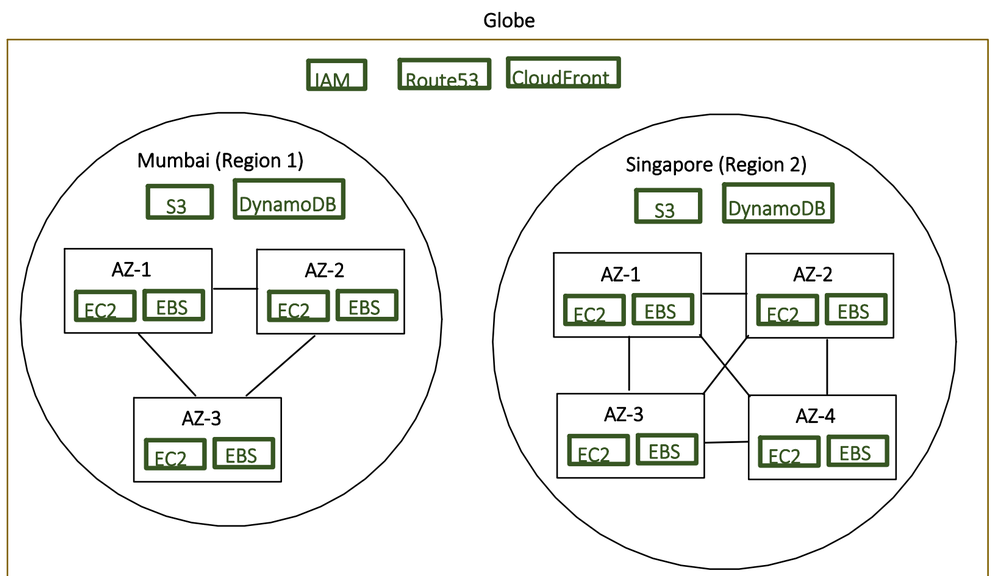
Global Services: means service shared across all Regions
- IAM,
- CloudFront (CDN)
- Route 53 (DNS)
  etc. are global services.

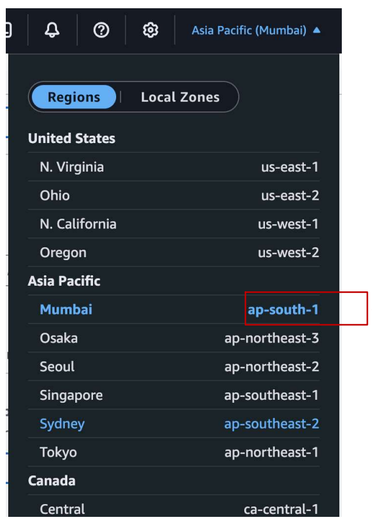


Now, if we further drill down within a Region:

- Some AWS services which are Regional are also Zonal, means services created in 1 AZ is not replicated to other AZ's in same Region.
  Example:
  Zonal Services:
  means services created in 1 AZ is not replicated to other AZ's in same Region
  - EC2,
  - EBS,
    etc.

Regional Services but not Zonal:
means Regional Services with AWS managed Multi-AZ availability, these services operate at region scope, and AWS automatically handles replication across multiple AZs.
- S3,
- DynamoDB
- Lambda
  etc.





{geography area} - {sub area} - {seq no}
Ex:
ap : Asia pacific
south: South Asia
1: 1st region in this sub area

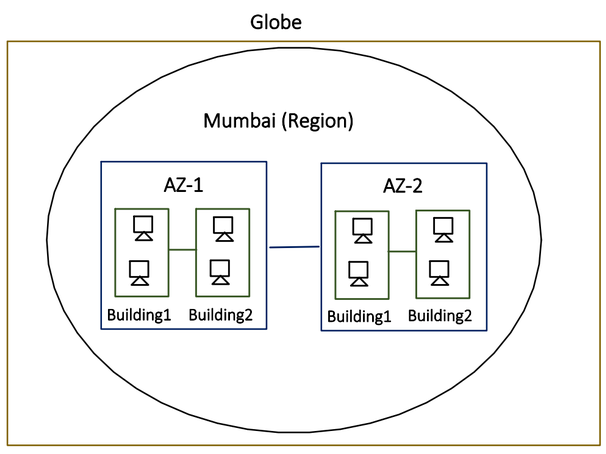
AWS has 30+ Regions distributed across the world.

Why Region exist:

- Latency:
  Closer = faster
  - If Users and Servers both are in India, we can provide low latency service.
  - If Users from India is served from Servers located in America, there will be latency add up for data travel time.
- Compliance:
  - Many countries like China / India etc. require certain data to stay in-country.
- Disaster Isolation:
  - Flood in Region1 will not affect Region2, both are completely independent and isolated (far distant).

2. Availability Zone

- Consider AZ as a building (or group of buildings) in which physical servers are kept.
- Physical servers are the one where all AWS services ultimately lives (whether they are Regional or Zonal).
- If service is Regional, than AWS automatically replicate it to other AZ physical servers.



Generally reach Region has >=3 AZ's.

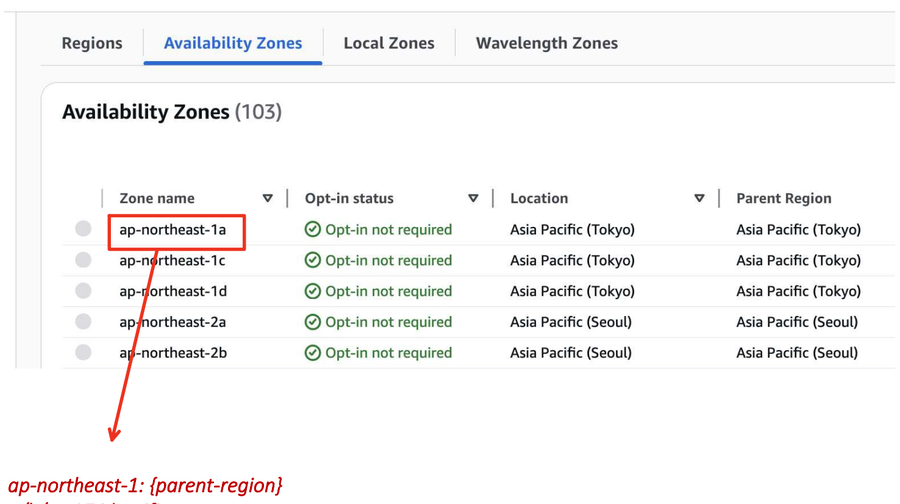
Reason:

Multiple AZ is required to survive single data center failure.

- If our app runs in only one AZ and that AZ goes down, we're offline. If it runs in two AZs, the other one keeps serving.

Note:

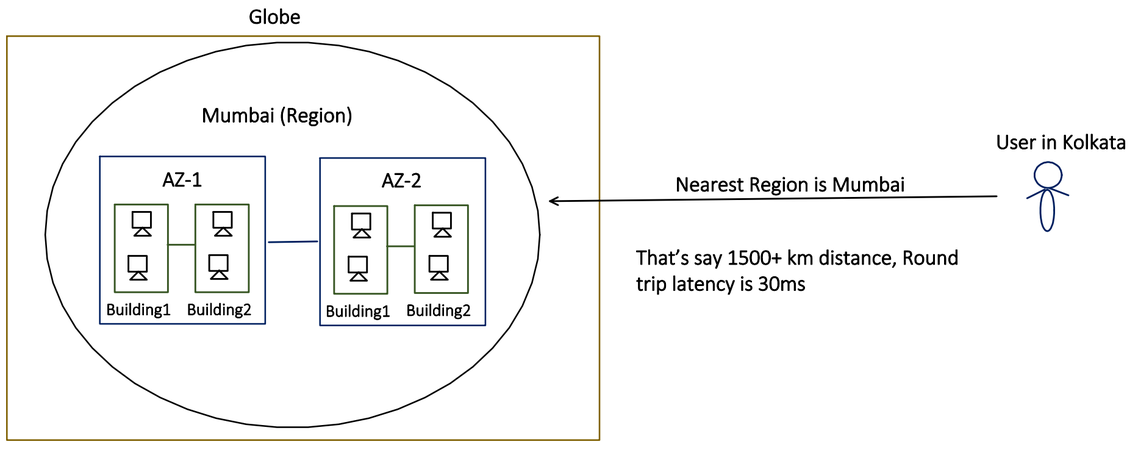
- 2 AZ are generally 10-100Km apart. Close enough to be fast and also far enough that one disaster can not hit both.
- Each AZs, power source, cooling, Network are independent.
- Two AZ's withing a Region are connected via private fiber cables, so that Round trip latency is in single digit millisecond (<1ms most of the time).



ap-northeast-1: {parent-region}
a/b/c - AZ identifier

3. Local Zone

Lets understand with below diagram:



30ms Latency is okay for most of the businesses, but say for business like Video calling, Live gaming etc. this is not acceptable.

So to reduce the Latency "Local Zone" concept is introduced.

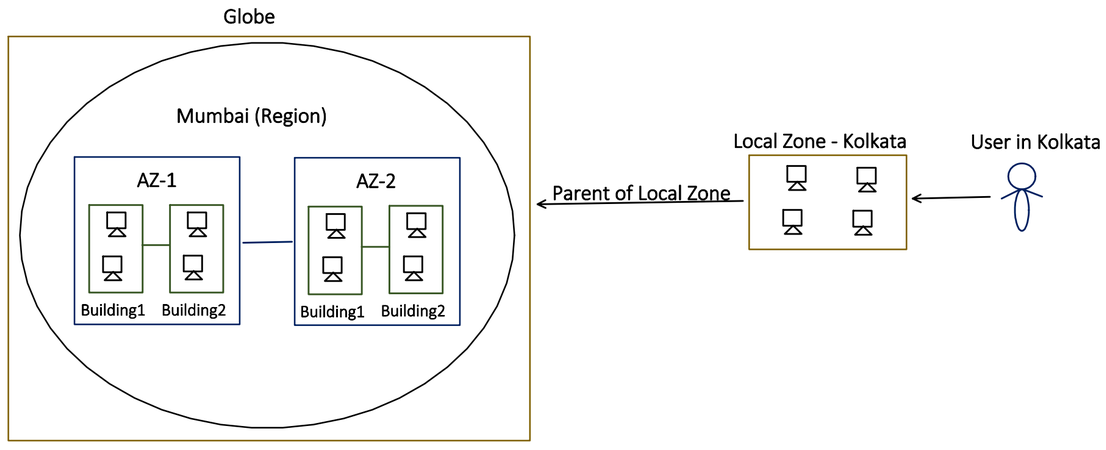
"Local Zone":

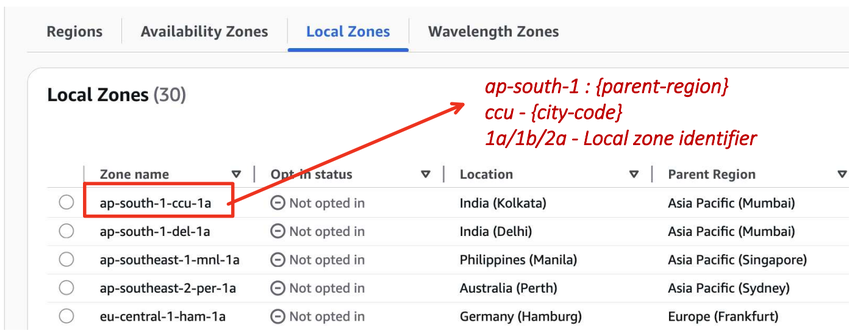
- Say Region is still Mumbai, but some servers I am running in Kolkata.
- So those small set of servers has limited service capabilities like:
  - EC2 - virtual machine
  - EBS - virtual hard disk

But rely on Parent region for rest of the services like S3, DynamoDB etc.

Suppose, we have a Use-case, where DB is accessed only once, after that its not required, in such scenarios this model shines.

But
If in each call our app need to talk to DB, then using Local Zone is of no use, as it has to connect with its Parent Region for the service.





4. Edge Location or Points of Presence (POP):

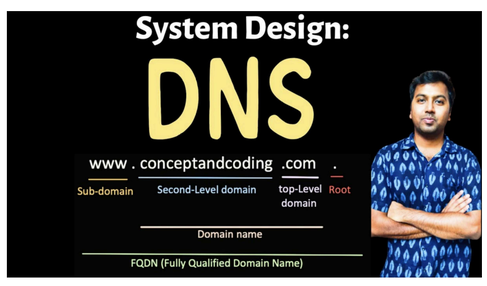
- Services running on these EDGE Locations are managed and operated by AWS.
- We do not deploy our servers there. All we can do is to make our service use these EDGE infrastructure.

But now question is, what is running in these EDGE locations?

CloudFront :
Its a CDN, which is used for Caching static data like Video, Images etc.

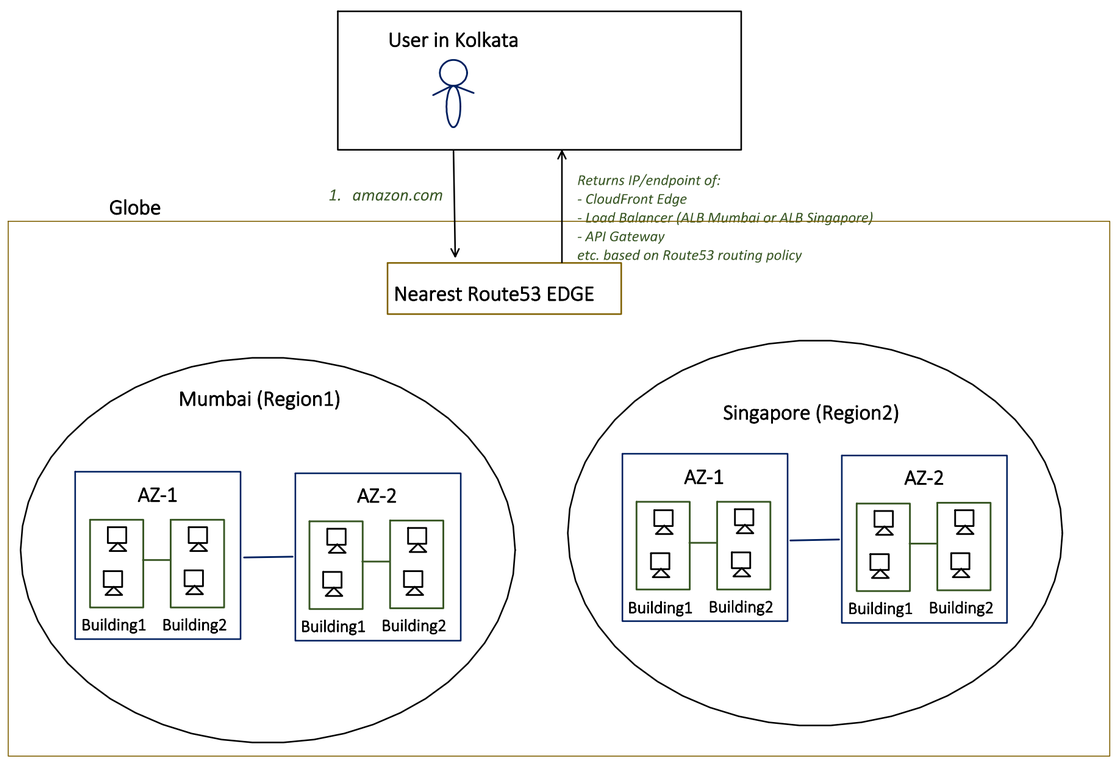
Route53:
Its a DNS, resolves DNS queries (amazon.com -> 54.x.x.x)

Covered DNS in HLD playlist,
check it out if there is any doubt



AWS Shield:
Provides protection from DDoS attack. It absorb network flood at EDGE location itself, before it reaches our Region.

etc.



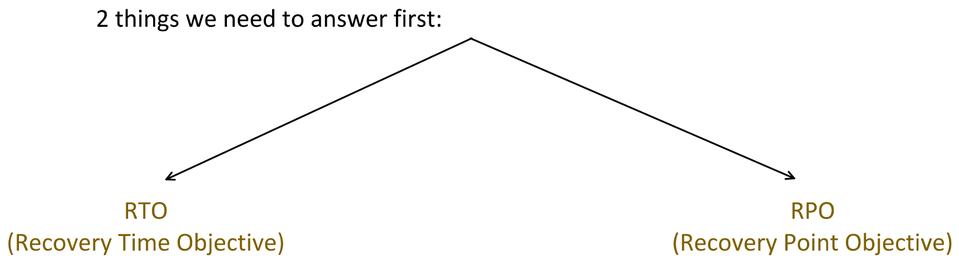
Disaster Recovery:

The entire AWS Global Infrastructure is designed with the assumption that failures are inevitable.

The goal is to minimize downtime and continue serving users with maximum availability and minimal disruption.

Now, when we design our system and need to handle disaster recovery part.

2 things we need to answer first:



How long can we be DOWN before we RECOVER?

How much data we can afford to loose?

Because based on above answer, we have to set up the infrastructure.

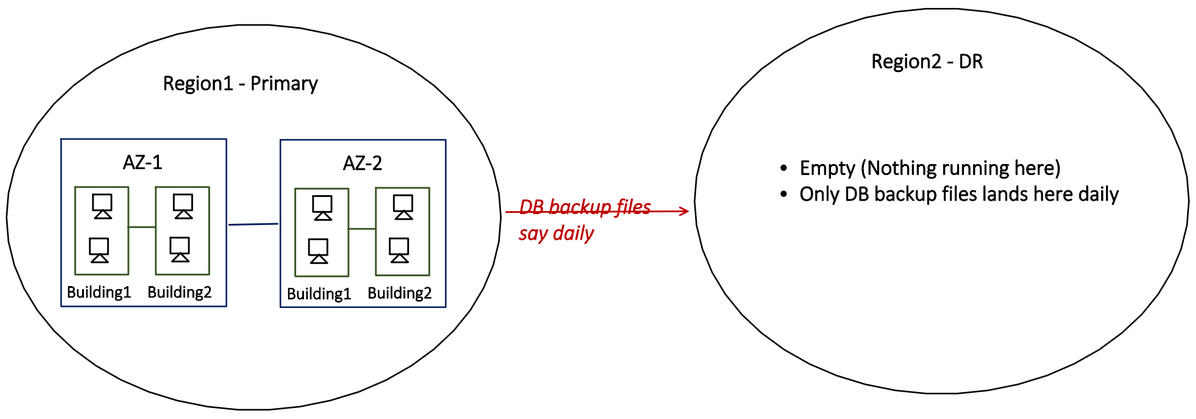
Smaller RTO = need more infrastructure, Replica Region should be up and running = More costly

Smaller RPO = need more frequent data replication (continuous) = More costly

So, we have various Disaster Recovery strategy (Cheapest to Most Expensive)

1. Backup and Restore:

- DR region is empty and do not takes any traffic.
  - No app server running
  - No DB instance running
- Just DB backup files present



On Disaster:

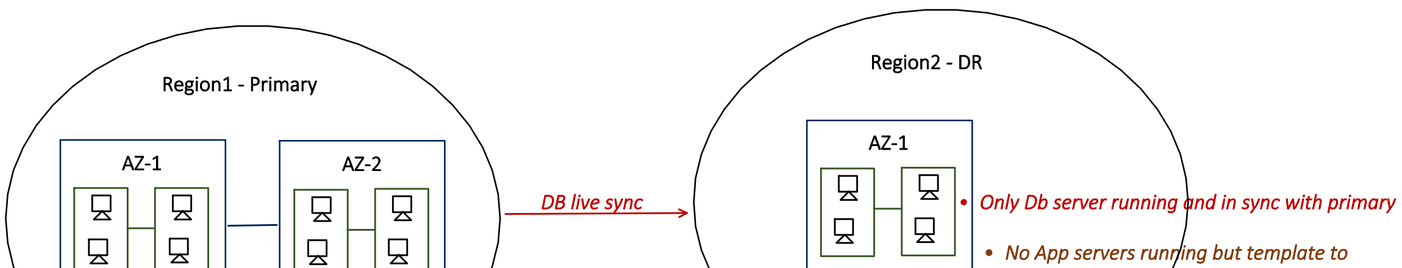
- We need to manually create app servers, db servers in Region2
- Restore the database from latest backup
- Promote Region2 DB as primary
- Repoint DNS to Region2

RTO=hours,
RPO=hours(depends on backup frequency).

Good for dev/test environments or low-traffic SaaS where a half-day outage is annoying but not catastrophic.

2. Pilot Live:

- In DR region keep the database alive (Continuous replicated from primary)
- But app server remains off.
- And DR region do not takes any traffic.



On Disaster:

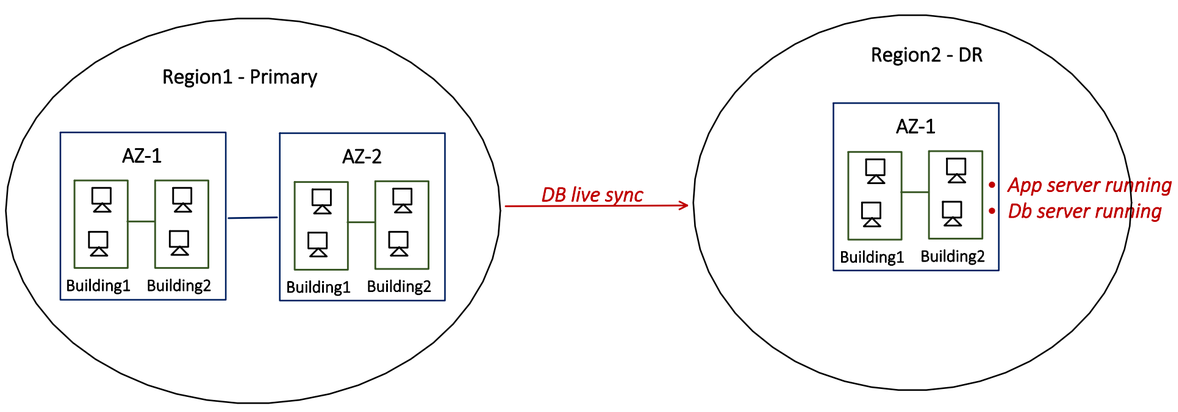
- Boot the app servers from pre-built templates in Region2
- Promote Region2 DB as primary
- Repoint DNS to Region2

RTO=10 to 30 minutes,
RPO= second to minutes (depends on replication lag).

3. Active-Passive (or Warm Standby):

- In DR region keeps:
  - App server alive : but at smaller scale, so during disaster all we need to do is scale up.
  - the database is alive (Continuous replicated from primary)
- And DR region do takes traffic:
  - Some follows: say 1% traffic served from DR region.

    Note:
    - *DR DB act as READ Replica - so Read traffic is serve from REGION2 (DR) database itself.*
    - *But if Write traffic comes to DR region, it invokes Primary Region DB instance.*
  - Some follows: 0% traffic from DR but time to time test DR region by making Primary region down and make DR as new Primary and serve all traffic from DR to make sure not surprises will encounter in case of actual Disaster.



On Disaster:

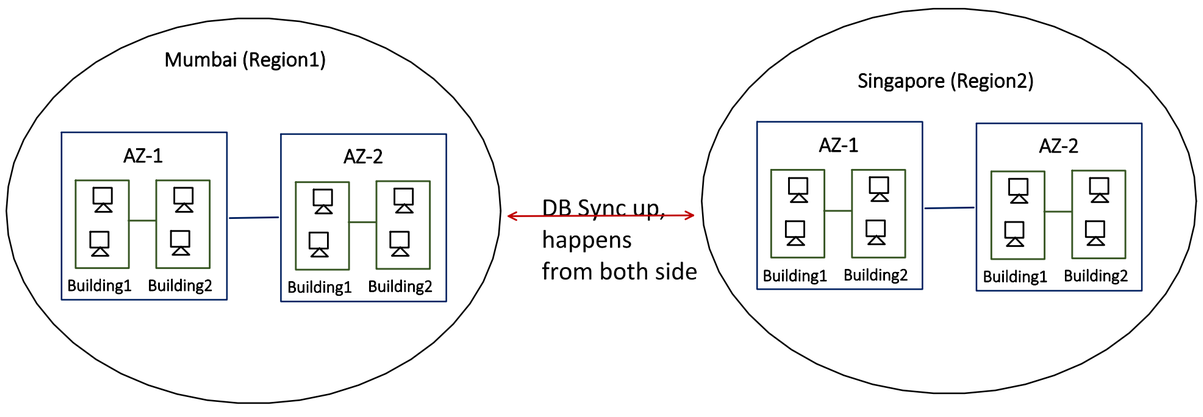
- Scale the app servers (say from 2 to 10)
- Promote Region2 DB as primary
- Repoint DNS to Region2

RTO= minutes ,
RPO= second to minutes (depends on replication lag).

4. Active-Active :

- No concept of Primary, DR anymore
- Both region serves the real users all the time.

If one fails, other absorb the traffic. So no downtime at all.



And there is a chance of conflict, as both region takes traffic:

Use case:

Available Balance = 100

Region1:
- Deposit in Balance = +10

Region2:
- Deposit in Balance = +20

Now, DB sync up happens and Conflict resolution needed?

How to resolve this DB conflict?

Last Write wins:

Like DynamoDB uses this approach:

Available Balance = 100

Region1:
Deposit in Balance at 10:00 = +10

Region2:
- Deposit in Balance at 10:01 = +20

So it will update the Available Balance = 120.

as Region2 write is latest. But this approach does not fit well in all scenarios like this one.

GEO Sharding:

Like Mumbai user data always go in Mumbai region.

So there is no chance of conflict as all writes and read for particular user always go in 1 region only.

Global database like AWS Aurora

- 1 region Database act as PRIMARY, which takes all WRITE requests.
- And other regions can serve READ requests from their DB but when WRITE request comes, they invoked Primary DB.